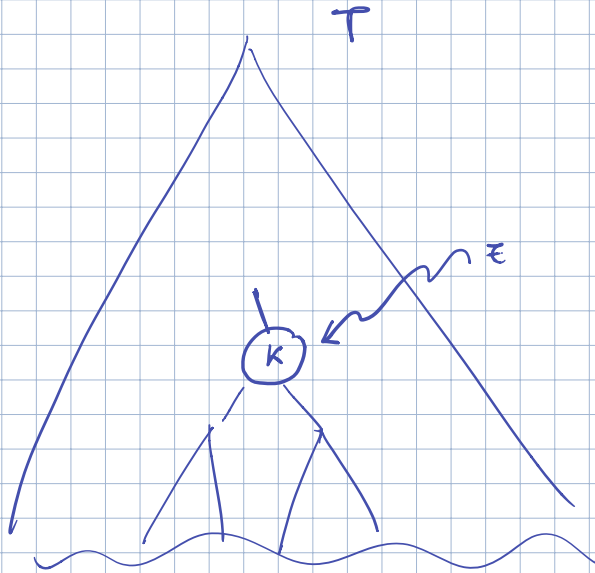


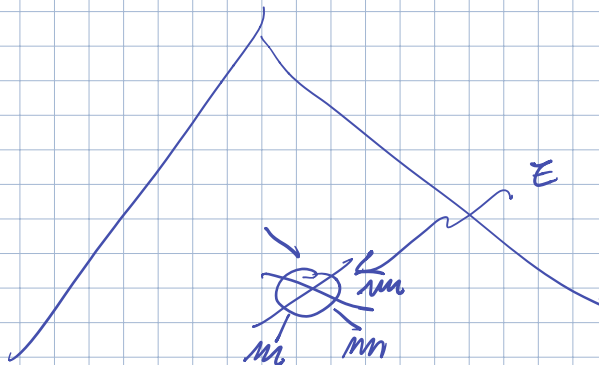
BST Cancellazione



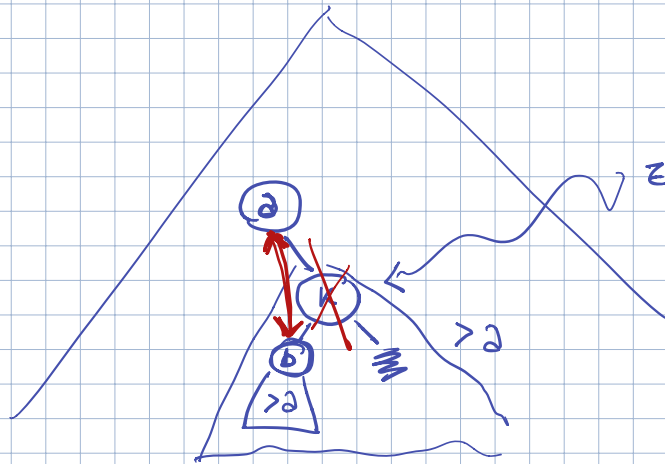
Rimuovere k da $T \rightarrow$ ottenendo ancora un BST

1) E non ha figli

Sostituire E
con NIL



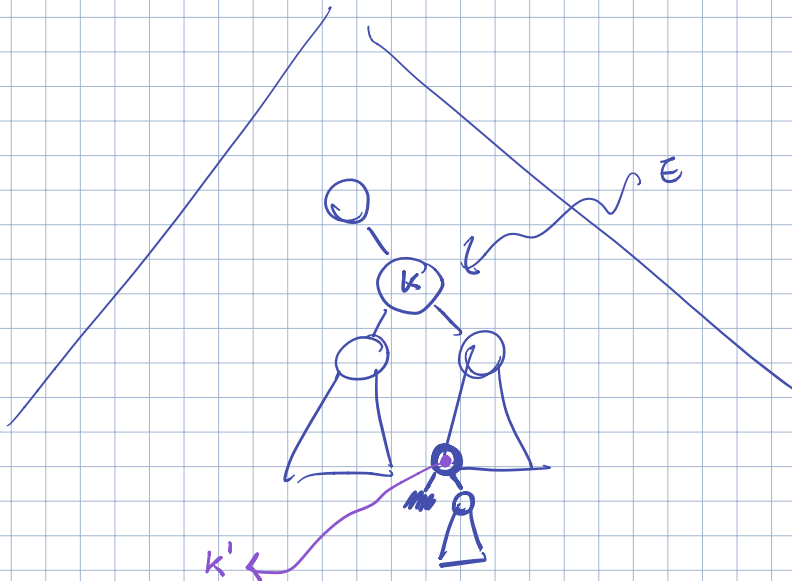
2) ε ha un figlio NIL e un figlio $\neq NIL$



Aggancio l'unico
figlio di ε
al genitore di z

$b > 2$

3) ε ha due figli $\neq NIL$



Idea: Cerco un nodo x t.c.:

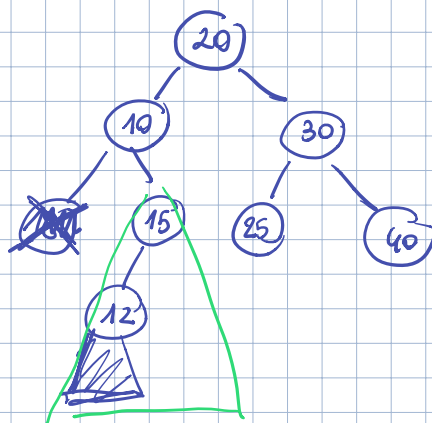
- la chiave di x può andare al posto di k
- è facile cancellare x perché x ha al più un figlio non NIL

Il min del figlio dx di z

Il successore di z

- Elimino il nodo x che contiene il successore di z (caso 1 o 2)
- Scrivo la chiave di x al posto della chiave di z

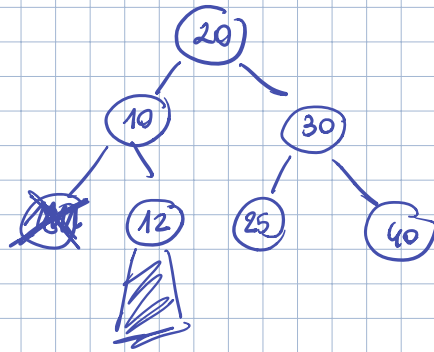
Esempio



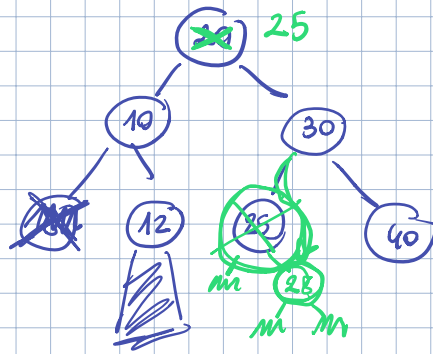
Cancellare 8

caso 1

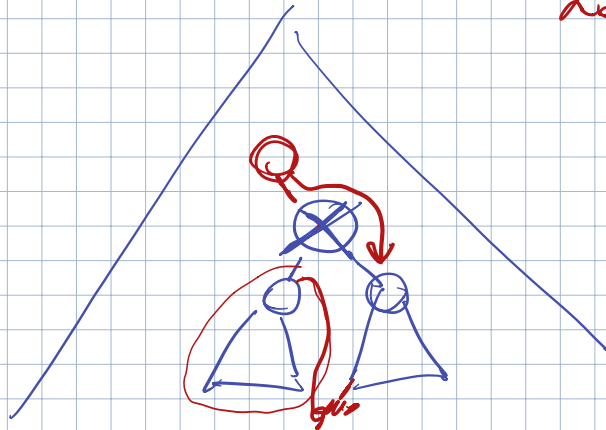
Cancellare 15



Cancellare 20



Proposta della chat



Lorenzo

Corrette

Stessa complessità

ma

Sbilancia
gli alberi!

Codice Eliminare la chiave di z da T

1) Individuo il nodo x che eliminerò dall'albero:

Caso 1 e Caso 2 $x \leftarrow z$

Caso 3 x è il successore di z

2) Elimino il nodo x sostituendolo con il suo figlio σ (figlio ev. non NIL di x)

3) Se $x \neq z$ copio la chiave di x in z

BST-Delete (T, z) { $z \neq \text{NIL}$

$O(h)$

$O(n)$

In-Place

1) if ($z.\text{left} = \text{NIL} \parallel z.\text{right} = \text{NIL}$) {
 $x \leftarrow z$
} else {
 $x \leftarrow \text{BST-successor}(z)$
}

$y \leftarrow x.\text{parent}$

if ($x.\text{left} \neq \text{NIL}$) {

$\sigma \leftarrow x.\text{left}$

} else {

$\sigma \leftarrow x.\text{right}$

2)

```

    }
    if (y = NIL) {
        T.root ← σ
    } else {
        if (x = y.left) {
            y.left ← σ
        } else {
            y.right ← σ
        }
    }

```

```

    if (σ ≠ NIL) {
        σ.parent ← y
    }

```

3)

```

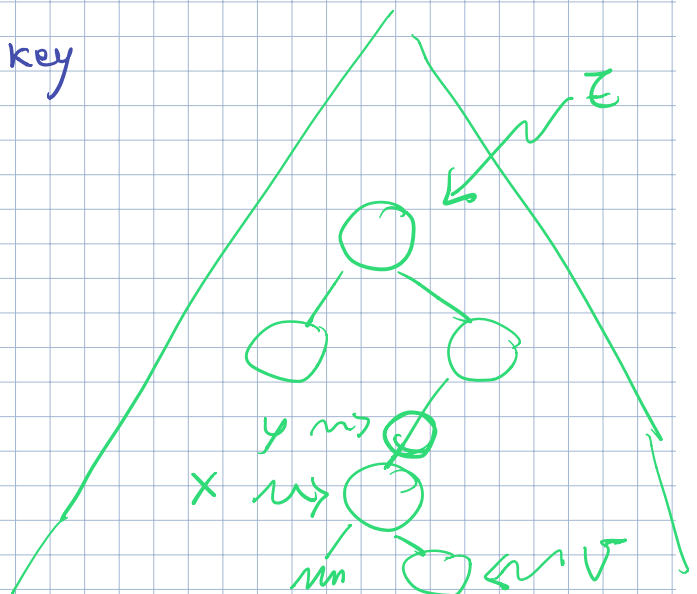
    if (z ≠ x) {
        z.key ← x.key
    }

```

```

}

```



Es.

Dato un vettore di interi A ^{no tip.} _{ordinato}
A sfruttando un BST

- Inserisco le n chiavi in T
- Faccio una visita in-order scrivendo in A

Sort(A) {

$T \leftarrow \text{Create_BST}(\dots)$

 for ($i \leftarrow 1$ to $A.\text{length}$) {

$x \leftarrow \text{Create_new_node}(A[i])$

 BST_Insert(T, x)

 }

 In_Order(T, A) $\leftarrow \Theta(n)$

}

} $O(n^2)$

Nel caso peggiore $\rightarrow \Theta(n^2)$
A è ordinato

$O(n^2)$

Nel caso migliore $\approx O(n \log n)$

In-place ?? No uso T

In-Order-V(T, A) { $\neq |T| = A.length$

$x \leftarrow T.root$

In-order-mode-V(x, A, 1)

}

In-order-mode-V(x, A, i) $\Theta(n)$

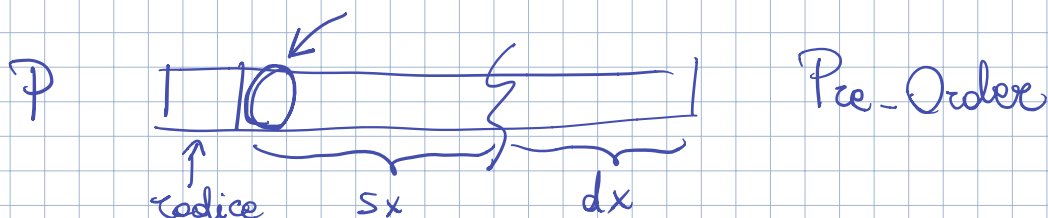
X CASA

$\uparrow$
modificato

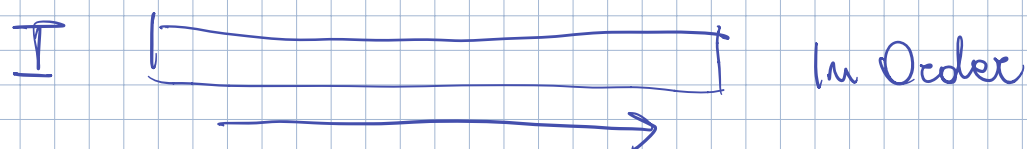
Esercizio

Sia P unettore contenente il risultato di una visita pre-order di un BST

Ricostruire il BST



Copie P in I e ordino I



Reconstruct_BST (P) {

$I \leftarrow \text{new_array}(P.\text{length})$

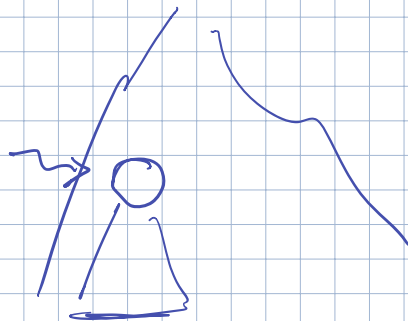
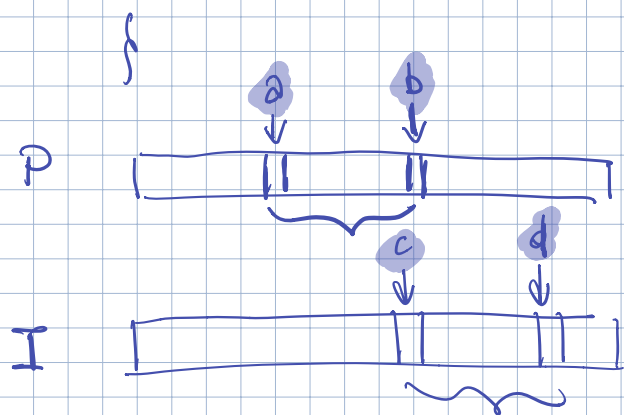
Copy (P, I) % copie $O(n)$

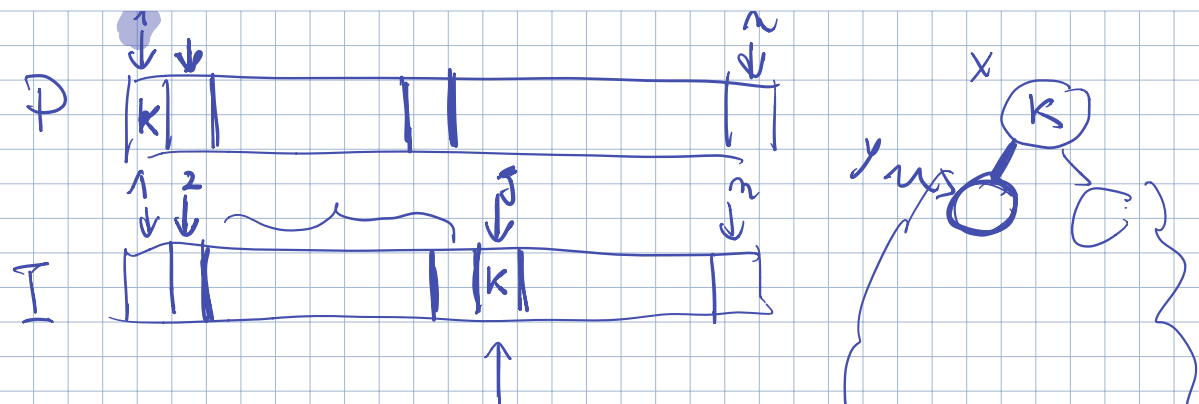
Sort (I) % $O(n \log n)$

$T \leftarrow \text{new_tree}()$

Reconstruct (P, I, T)

return T





Reconstruct (P, I, $\overset{P}{a}, \overset{P}{b}, \overset{I}{c}, \overset{I}{d}, x$) {

$x.key \leftarrow P[a]$

if ($a < b$) {

$O(\log n) \Rightarrow j \leftarrow \text{Search}(I, c, d, \underline{P[a]})$

if ($j > c$) {

$y \leftarrow \text{new-node}()$

$y.parent \leftarrow x$

$x.left \leftarrow y$

Reconstruct (P, I, $a+1, \overset{a+j-c}{\uparrow}, c, j-1, y$)

}

return x